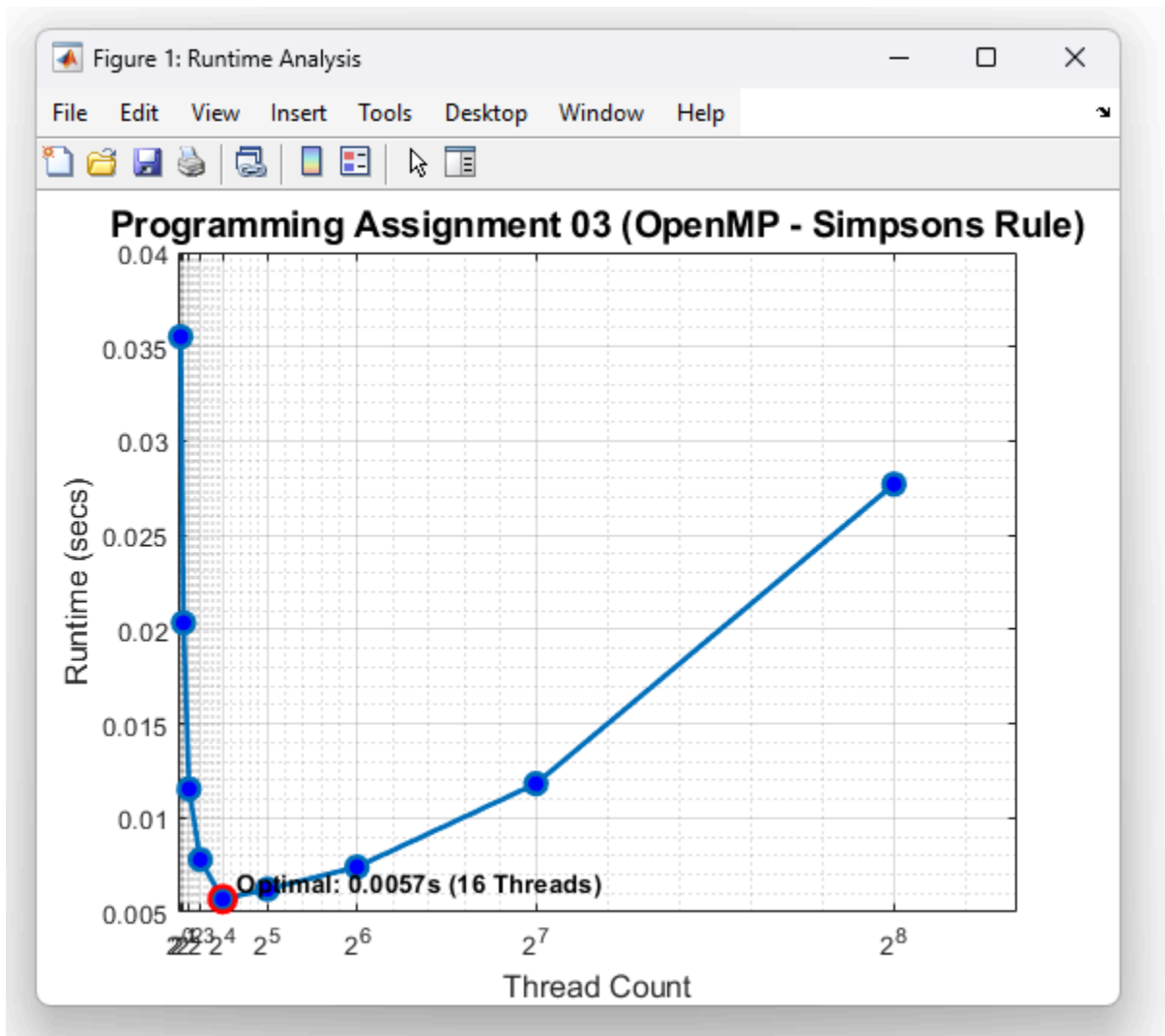


Brandon Nguyen, 827813045
COMPE596 - Spring 2026
2/13/2026

COMPE596 P03 - OpenMP Simpson's Quadrature Method

1) Runtime Plot



2) Code Listing

// Main Program

```
#include <omp.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
```

```

#include <time.h>
#include <math.h>

double f(double x) {
    return acos(cos(x) / (1.0 + 2.0 * cos(x)));
}

int main() {

    long n = 1000000;
    const double PI = acos(-1.0);
    double a = 0.0;
    double b = PI / 2.0;
    double h = (b - a) / n;

    const double exact_solution = (5.0 * PI * PI) / 24.0;

    int thread_counts[] = {1, 2, 4, 8, 16, 32, 64, 128, 256};
    int num_tests = 9;

    omp_set_dynamic(0);

    printf("Threads Time (sec) Result Error\n");

    for (int t = 0; t < num_tests; t++) {

        int Nthrds = thread_counts[t];
        omp_set_num_threads(Nthrds);

        double start_time = omp_get_wtime();
        double sum = 0.0;

        #pragma omp parallel for reduction(+:sum)
        for (long i = 1; i < n; i++) {

            double x = a + i * h;

            if (i % 2 == 0)
                sum += 2.0 * f(x);
        }
    }
}

```

```

        else
            sum += 4.0 * f(x);
    }

    double integral = (h / 3.0) * (f(a) + f(b) + sum);
    double end_time = omp_get_wtime();

    double error = fabs(integral - exact_solution);
    double duration = end_time - start_time;

    printf("%7d %10.6f %.15f %.3e\n",
        Nthrds, duration, integral, error);
}

return 0;
}

```

// Makefile

```

SHELL := bash
CC ?= gcc-13
CFLAGS ?= -O3 -std=c11 -Wall -Wextra
OMPFLAG ?= -fopenmp
LDFLAGS ?= -lm

TARGET = P03

.PHONY: all clean run

all: $(TARGET)

$(TARGET): P03.c
    $(CC) $(CFLAGS) $(OMPFLAG) -o $$@ $< $(LDFLAGS)

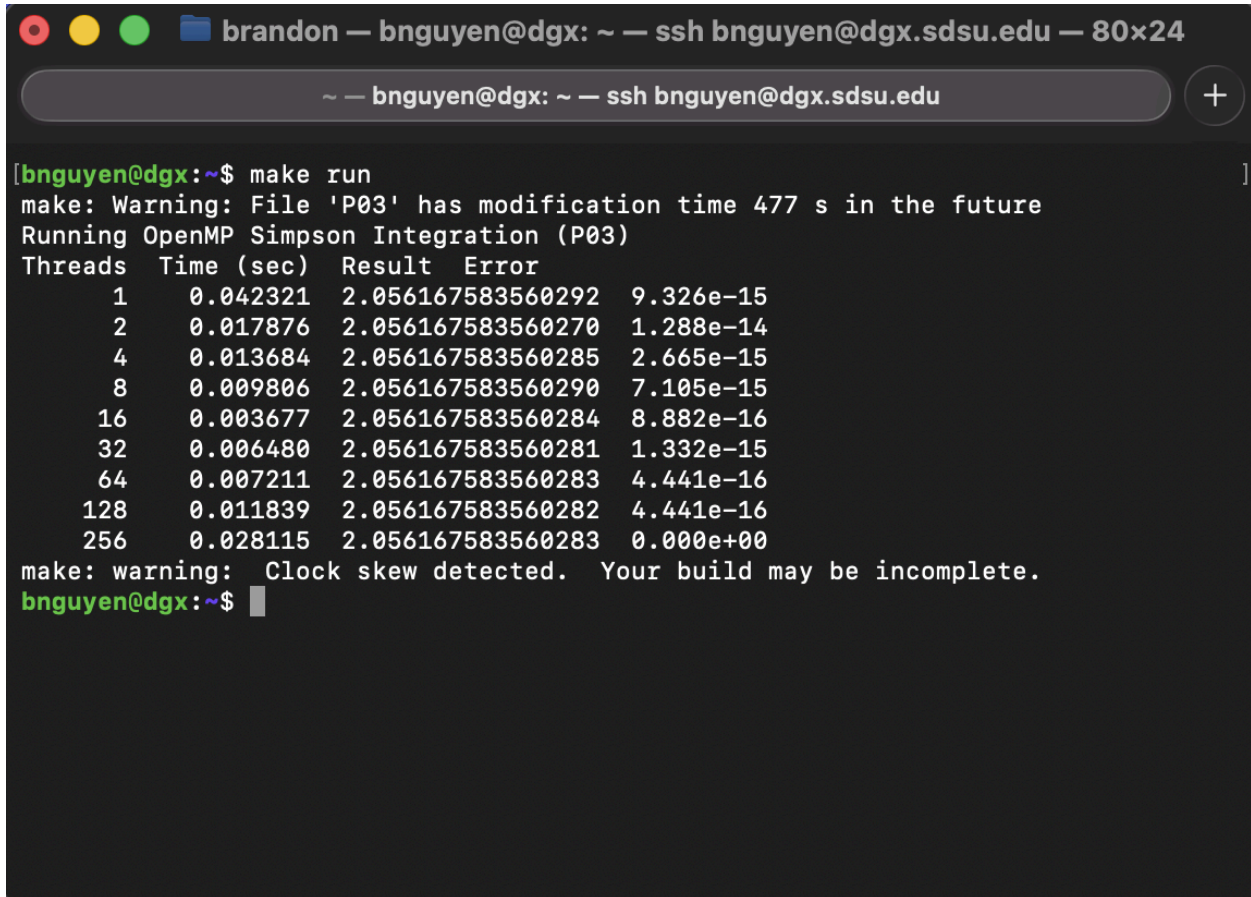
run: $(TARGET)
    @export OMP_PLACES=threads; \
    export OMP_PROC_BIND=true; \
    export OMP_NUM_THREADS=128; \
    echo "Running OpenMP Simpson Integration (P03)"; \
    ./$(TARGET)

```

clean:

```
rm -f $(TARGET)
```

3) Program Output



A terminal window titled "brandon — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu — 80x24". The terminal shows the execution of a program using OpenMP for Simpson Integration. The output includes a warning about a future modification time for file 'P03', followed by a table of results for different thread counts. The results show that as the number of threads increases, the execution time decreases and the error approaches zero. A final warning about clock skew is also present.

```
[bnguyen@dgx:~$ make run
make: Warning: File 'P03' has modification time 477 s in the future
Running OpenMP Simpson Integration (P03)
Threads   Time (sec)   Result      Error
    1      0.042321  2.056167583560292  9.326e-15
    2      0.017876  2.056167583560270  1.288e-14
    4      0.013684  2.056167583560285  2.665e-15
    8      0.009806  2.056167583560290  7.105e-15
   16      0.003677  2.056167583560284  8.882e-16
   32      0.006480  2.056167583560281  1.332e-15
   64      0.007211  2.056167583560283  4.441e-16
  128      0.011839  2.056167583560282  4.441e-16
  256      0.028115  2.056167583560283  0.000e+00
make: warning: Clock skew detected. Your build may be incomplete.
bnguyen@dgx:~$
```

4) MATLAB Plot Script

```
close all;
```

```
clc;
```

```
threads = [1 2 4 8 16 32 64 128 256];
```

```
runtime = [0.035529 0.020354 0.011536 0.007785 0.005694 ...
           0.006219 0.007398 0.011813 0.027707];
```

```
figure('Name', 'Runtime Analysis');
```

```
plot(threads, runtime, 'o-', ...
```

```
    'LineWidth', 2, ...
```

```
    'MarkerFaceColor', 'b', ...
```

```

'MarkerSize', 8);

set(gca, 'XTick', threads);
set(gca, 'XTickLabel', ...
    arrayfun(@(x) sprintf('2^{%d}', round(log2(x))), ...
    threads, 'UniformOutput', false));

xlabel('Thread Count', 'FontSize', 12);
ylabel('Runtime (secs)', 'FontSize', 12);
title('Programming Assignment 03 (OpenMP - Simpsons Rule)', ...
    'FontSize', 14);

grid on;
set(gcf, 'Color', 'white');
ax = gca;
ax.YMinorGrid = 'on';
ax.XMinorGrid = 'on';

% Highlight optimal point
[min_time, min_idx] = min(runtime);

hold on;
plot(threads(min_idx), min_time, 'ro', ...
    'MarkerSize', 10, 'LineWidth', 2);

text(threads(min_idx), min_time, ...
    sprintf(' Optimal: %.4fs (%d Threads)', ...
    min_time, threads(min_idx)), ...
    'VerticalAlignment', 'bottom', ...
    'FontWeight', 'bold');

```

5) Discussion

In P03, we are tasked with writing a program using the DGX server to observe OpenMP parallel for and also parallel reduction, as demonstrated in class, to speed up the composite 1/3 Simpson's quadrature method. Once again, we are able to use a bash script driver in Makefile to help run the main program. Lastly, we are then required to plot our terminal output results in MATLAB in order to precisely analyze and compare between runtime (in seconds) and thread count to locate the optimal value. After writing the Simpson's Method program, I tested a range of thread counts ranging from 2^0 to

2^8 to give a good idea of runtime differences. According to my findings in the graphs above and my device's CPU specs, the optimal thread count appears to be 16 threads for my program before the system or program hits a saturation point and the processing time goes back up as the thread count continues to rise. This third programming assignment continued expanding our knowledge in accelerated computing by teaching us new syntax, formulas, and solidified our implementation of using Makefiles for compiling convenience.